Python Modules

Python modules are files that contains reusable Python code.
They help organize and structure program by grouping related functions, classes, and variables together.
Using modules makes your code easier to maintain, extend, and share.

What is a Module?

A module is simply a Python file (with a .py extension) that contains definitions of functions, classes, and variables.
Any Python file can be imported as a module.

Creating a Module

Creating a module is as simple as saving Python code in a `.py` file:

```python
# mymodule.py
def greet(name):
    return f'Hello, {name}!'

PI = 3.14159

class Calculator:
    def add(self, a, b):
        return a + b
```

Importing Modules

There are several ways to import modules:

1. Basic import:

```python
import mymodule
result = mymodule.greet('Alice')
```

2. Import with alias:

```python
import mymodule as mm
result = mm.greet('Alice')
```

3. Import specific items:

```python
from mymodule import greet, PI
result = greet('Alice')
```

4. Import everything (not recommended):

```
from mymodule import *
```

Types of Modules

- **Built-in modules**: Come with Python (like `math`, `os`, `sys`, `datetime`)
- **Third-party modules**: Installed via `pip` (like `numpy`, `pandas`, `requests`)
- **Custom modules**: your own `.py` files

Module Search Path

When you import a module, Python searches for it in the following order:

1. The current directory
2. Directories in the `PYTHONPATH` environment variable
3. Installation-dependent default directories

You can check the search path:

```
import sys
print(sys.path)
```

The `__name__` Variable

Every module has a special built-in variable called `__name__`.
If the module is run directly, `__name__` equals `'__main__'`.
This allows you to control which code runs when the file is executed versus imported.

```
# mymodule.py
def main():
    print('Running as main program')

if __name__ == '__main__':
    main()
```

Packages

A package is a collection of modules organized in directories.
A directory is treated as a package if it contains an `__init__.py` file.

```
mypackage/

    __init__.py

    module1.py
```

```
module2.py

subpackage/

    __init__.py

    module3.py
```

Import from packages:

```python
from mypackage import module1
from mypackage.subpackage import module3
```

Module Attributes

All modules have several special attributes:

- `__name__`: Module's name
- `__file__`: Path to the module file
- `__doc__`: Module's docstring
- `__dict__`: Module's namespace as a dictionary
- `__package__`: Package the module belongs to

Reloading Modules

Modules are only loaded once per interpreter session.
if you modify a module and want to reload it without restarting Python:

```python
import importlib
importlib.reload(mymodule)
```

The `dir()` Function

Use `dir()` to see what names a module defines:

```python
import math
print(dir(math))  # Lists all functions and variables in math module
```

Module Best Practices

- Modules should have **clear**, **descriptive name**s in lowercase.
- Use **underscores** for multi-word names (like `my_module.py`).
- Include a **docstring** at the top of each module explaining its purpose.
- Avoid **circular imports** where two modules import each other.
- Keep modules **focused and cohesive**, following the single responsibility principle.

Lecture 09

Common Built-in Modules

Python comes with many useful modules including

- `os` for operating system interactions
- `sys` for system-specific parameters and functions
- `math` for mathematical functions
- `random` for random number generation
- `datetime` for date and time handling
- `json` for JSON encoding/decoding
- `re` for regular expressions
- `collections` for specialized container datatypes

Installing Third-Party Modules

Use **pip**, Python's package installer:

```
pip install package_name

pip uninstall package_name

pip list  # Show installed packages
```

Python's module system is what makes it powerful and extensible, allowing you to organize code logically, reuse code across projects, and leverage the vast ecosystem of third-party libraries.

Summary

- A **module** is any .py file that contains reusable Python code.
- Modules make programs modular, organized, and easier to maintain.
- Python supports **built-in**, **third-party**, and **custom** modules.
- The `__name__ == '__main__'` construct controls script execution.
- **Packages** organize multiple modules into structured directories.
- Use `dir()` to inspect modules and `importlib.reload()` to reload them.
- Follow **best practices** for naming, documentation, and modular design.

In essence: Modules are the *foundational building blocks* of all Python applications - they encourage code reuse, logical organization, maintainability, and scalability.